Research Progress Report: PROBE-CASCADE Data Layer, Baselines, and IRT Onboarding (M0 to M3)

Multi-LLM Comparison Platform: From Model Comparison to a Cost-Aware Router That Generalises to New Models

Student: Venu Dattathreya Vemuru (811776500)
Course: CSCI 7200, Master's Project
Report period: 1 October to 30 October 2026 (since Research Progress Report 7)
GitHub: https://github.com/venu284/LLM_Comparison.git

---

1. Summary

This report covers milestones M0 to M3 of the PROBE-CASCADE plan from Report 7: setup, the data layer, the replay harness with baselines, and the IRT model for onboarding new models. All results are offline, on the public CodeRouterBench release (8 frontier models, 7,080 probing tasks, 2,919 test tasks). No model APIs were called.

Main results:

1. **The paper's published numbers reproduce exactly** from the released data: the oracle (57.00), always using Claude Opus (43.83 AvgPerf, 1.29 Perf/$) and the released ACRouter decisions (50.14). This required finding that the paper scores an older, incomplete matrix with 881 missing results counted as failures.
2. **Simple baselines can match ACRouter.** Under our definitions, which are not the paper's implementation (metadata features; LinUCB warm-started on the probing split), DimensionBest scores 50.10 and LinUCB 50.32. The paper reports 47.50 and 46.84 for its versions and 50.14 for ACRouter.
3. **A cheap-first cascade works on a frontier pool.** Qwen3-Max with escalation to Opus scores 49.4 AvgPerf at 44% lower cost than Always-Opus, and stays above Opus's acceptance rate even when the verifier is wrong 20% of the time.
4. **A new model can be profiled from about 50 tasks.** With 50 random probe tasks the router recovers 84% of the predictive gain of the full 7,080-task profile and ranks the eight models with Spearman $\rho = 0.81$ (1PL) and 0.88 (2PL), passing the M3 gate.
5. **Fisher-selected probe tasks did worse than random ones** on the real data at every probe size, although they win on simulated one-dimensional data. The likely cause is that model ability differs by dimension. This is a negative result for the anchor-selection claim as built, and it sets the first task of M4.

---

2. What Was Built

Milestone	Delivered (in `router/`)
M0 Setup	Docker check, package skeleton, test configuration, novelty check, reference check
M1 Data	`data/coderouterbench.py` (two views: the complete release and the paper's matrix), `data/prompts.py` (rebuilds prompt text, which the release omits), `data/prices.py`, `data/scoring.py`
M2 Replay and baselines	`sim/metrics.py`, `sim/verifier.py` (simulated verifier with error rate ε), `sim/stream.py`, `baselines/` (always-one-model, random, DimensionBest, SourceBest, kNN, LinUCB, fixed cascade, released ACRouter decisions)
M3 IRT and onboarding	`core/irt.py` (1PL/2PL fit with priors), `core/anchors.py` (Fisher and random probe selection), difficulty predictor in `experiments/m3_difficulty.py`, simulation check in `experiments/m3_fisher_synthetic.py`

The experiment scripts are `router/experiments/m2_reproduce.py`, `m3_irt.py` and `m3_difficulty.py`. Their outputs are in `router/experiments/out/`. There are 19 router tests, all passing, including one that checks the reproduced published numbers and one that checks Fisher selection on simulated data.

---

3. Results

3.1 Data checks (M1)

Check	Expected	Observed
Rows (probing / test)	56,640 / 23,352	56,640 / 23,352
Missing scores	0	0
Rows with no token record	106 / 42	106 / 42
Missing cells in the paper's matrix (test)	not published	881
Prompt text recovered	target 95%	**48%** (5 of 11 source benchmarks)

The release ships no prompt text for its in-distribution tasks. We recovered it for DS-1000, CRUXEval, MBPP, HumanEval and BigCodeBench (4,797 tasks), which covers six of the nine dimensions. 41% of scores are partial rather than pass/fail, so every analysis below counts a task as solved when its score is at least 0.5.

3.2 Baseline reproduction (M2)

ID test split, 2,919 tasks. "Paper view" scores the original matrix with missing results counted as failures; "complete" uses the released, filled-in matrix. kNN and LinUCB use one-hot dimension and source labels as features, because most tasks have no prompt. LinUCB is warm-started on the 7,080 probing tasks and then run online over the test stream. Online baselines are averaged over 5 seeds.

Router	Paper	Ours, paper view	Ours, complete	Perf/$ (paper view)
Oracle	57.00	**57.00**	60.02	4.21
ACRouter (released decisions)	49.98	**50.14**	50.14	2.25
LinUCB	46.84	50.32	51.92	2.94
DimensionBest	47.50	50.10	51.68	4.02
kNN retrieval	47.18	48.45	50.39	4.72
Always-Opus	43.83	**43.83**	43.83	1.29
Random (10 seeds)	38.75	38.35	39.28	2.72

Bold numbers match the paper exactly; ACRouter matches its released metrics file (the paper text says 49.98). Cumulative regret also matches (Always-Opus 387.4 against 387.1; ACRouter 202.0 against 202.0 in the release).

The M2 gate asked for every baseline within ±1.0 point. Oracle, ACRouter, Always-Opus and Random pass. DimensionBest, kNN and LinUCB do not: ours are 1.3 to 3.5 points higher. The paper's baseline code is no longer public (its GitHub repository returns 404), and our kNN and LinUCB use dimension and source labels because the release has no prompts for most tasks. Our numbers are therefore not the paper's implementation. They do show that a per-dimension lookup table, fitted on the probing split, routes as well as the released ACRouter at 56% of its cost.

3.3 Cost of a cheap-first cascade (M2)

Complete matrix. The acceptance floor is Always-Opus's rate of solved tasks (43.5%). The cascade escalates only on the simulated verifier's verdict, never on the true score; ε is the verifier's error rate. Values are the mean of 5 seeds. The interval is a bootstrap 95% CI over tasks, taken from the first seed.

Policy	AvgPerf (95% CI)	Solved %	$Total	Cost per solved task	Cost vs Always-Opus
Always-Opus	43.8 (42.3 to 45.3)	43.5	34.02	$0.0268	0%
Always-GPT-5.4	42.2 (40.8 to 43.7)	43.3	16.57	$0.0131	−51%
Qwen3-Max → Opus, ε = 0	49.4 (47.9 to 50.9)	51.8	19.14	$0.0127	−44%
Qwen3-Max → Opus, ε = 0.2	46.2 (45.2 to 48.1)	46.8	19.12	$0.0140	−44%
Qwen3-Max → GPT-5.4, ε = 0	46.3 (44.8 to 47.8)	48.9	8.99	$0.0063	−74%
Qwen3-Max → GPT-5.4, ε = 0.2	44.0 (42.7 to 45.6)	45.1	9.35	$0.0071	−73%
Qwen3-Max → GPT-5.4 → Opus, ε = 0	50.4 (49.0 to 52.0)	54.3	24.13	$0.0152	−29%
DimensionBest	51.7 (50.2 to 53.2)	52.5	13.09	$0.0085	−62%

Every cascade clears the floor at 29% to 74% lower cost, including at 20% verifier error. This supports the sufficiency premise of the project on a priced frontier pool. Two caveats apply. The cost of running the verifier itself (tests, sandbox) is not counted. And DimensionBest, which needs no verifier, is more accurate than every fixed cascade and cheaper than all but Qwen3-Max → GPT-5.4. A learned router has to beat both.

3.4 Choosing the IRT model (M3)

Held-out log-likelihood per cell (10% of cells hidden, 3 seeds; higher is better).

Model	All dimensions	Mostly pass/fail dimensions
Per-model pass rate, no task information	−0.662	−0.611
1PL, one ability per model	−0.373	−0.383
2PL, one ability per model	−0.366	−0.381
1PL, one ability per model and dimension	**−0.307**	**−0.338**

Task difficulty explains most of the outcome. A model's ability differs by dimension more than a second item parameter helps, so M4 will use a separate ability per dimension.

3.5 Predicting difficulty from the prompt (M3)

On the 4,797 tasks with recovered prompts (2,990 distinct problems), 5-fold cross-validation with folds grouped by prompt. Variants of one problem (for example a HumanEval task and its completion, test-generation and other-language versions) never appear in both training and test folds. The target is each

task's fitted difficulty. "Pass AUC" checks how well the predicted difficulty, combined with each model's ability, predicts the real pass/fail outcomes.

Predictor	Spearman ρ with difficulty	Pass AUC
Dimension average	0.45	0.75
Source-benchmark average	0.58	0.81
MiniLM prompt embedding + ridge	0.57	0.78
Embedding + source label + ridge	**0.77**	**0.89**
Fitted difficulty (ceiling)	1.00	0.97

The embedding alone is no better than knowing the source benchmark. Combined with it, the embedding adds real information (ρ 0.77 against 0.58) on problems it has never seen, so a new task's difficulty can be estimated before any model runs on it.

3.6 Onboarding a new model with k probe tasks (M3)

Leave-one-model-out: fit the task parameters on 7 models, then estimate the held-out model's ability from k probe tasks. 8 held-out models × 10 seeds, 1PL. "Gain recovered" is the main measure: the share of the 2,919 test tasks' log-likelihood improvement that the full 7,080-task profile achieves over knowing nothing (k = 0). Ability error is the distance to the ability fitted from the full profile, with 95% CIs.

k	Gain recovered, random	Gain recovered, Fisher	Ability error, random	Ability error, Fisher
0	0%	0%	0.79	0.79
5	34%	14%	0.55 (0.47 to 0.63)	0.61 (0.51 to 0.70)
10	57%	28%	0.44 (0.37 to 0.51)	0.56 (0.47 to 0.65)
25	73%	58%	0.34 (0.28 to 0.39)	0.43 (0.36 to 0.50)
50	84%	71%	0.27 (0.23 to 0.31)	0.35 (0.30 to 0.41)
100	93%	81%	0.19 (0.16 to 0.22)	0.30 (0.25 to 0.34)

Ranking check (M3 gate: Spearman ρ ≥ 0.8 between the k = 50 ability and the full pass rate):

Anchors at k = 50	1PL	2PL
Random	**0.81**	**0.88**
Fisher (adaptive)	0.62	0.50

The gate passes with random anchors. The ability estimate settles quickly: 25 tasks recover nearly three quarters of the full profile's value.

Fisher-selected anchors, the method from computerised adaptive testing that we planned to contribute, lose to random selection at every k. A simulation checks that this is not a code error. On synthetic one-dimensional 2PL data with the same number of tasks, Fisher anchors beat random ones at k = 50. With the true task parameters the ability error is 0.09 against 0.28. With parameters fitted from only seven

respondents, as in the real experiment, it is 0.24 against 0.32 (`m3_fisher_synthetic.py`). So noisy calibration alone does not explain the real-data result. The remaining explanation is the one §3.4 points to: real abilities differ by dimension, and a single ability lets adaptive selection pull its anchors from whichever dimensions look most informative, which misrepresents the model elsewhere. Building the simulation also exposed a numerical bug: the ability solver could diverge on highly discriminating anchors. It is now damped and covered by a test. The real-data numbers above were unchanged by the fix.

---

4. What the Results Change

- **Contribution.** "Fisher anchors beat random anchors" is not supported on real data with a single ability per model. M4 will test Fisher selection within each dimension, with per-dimension ability, before we decide whether the anchor claim survives. If it does not, the contribution rests on the onboarding curve inside a verified cascade (§3.6, §3.3) and on live onboarding of free-tier models.
- **Model.** M4 uses per-dimension abilities (§3.4) and the embedding + source difficulty predictor (§3.5).
- **Baselines.** DimensionBest and LinUCB become the main baselines to beat, not Always-Opus. They match ACRouter on the paper's own data (§3.2).
- **Data.** Prompt recovery for the remaining six source benchmarks (LiveCodeBench, DebugBench, CodeXGLUE, Bugs2Fix, CanItEdit, SWE-bench) is needed before results can cover all dimensions.

5. Corrections to Report 7

- API-013 and CSS-012 are not defective. Their reference solutions pass 12/12 and 8/8 tests. Why the models' code collected no tests is still open.
- After a taxonomy fix, `drop_artifactual` drops 21 rows, not 20. RQ1 is p = 2.4e-4 and V = 0.267. The conclusion holds.
- The claim to be "the first measurement of how many probe tasks a new model needs" is retracted. UniRoute (2502.08773) and ICL-Router (2510.09719) publish such curves.
- CodeRouterBench has 9 in-distribution dimensions, not 10.
- The released ACRouter does escalate on its out-of-distribution set, through a fixed cascade.
- The ACRouter code repository is no longer public. Only the data and result files are.
- Seven reference titles in Report 7 were wrong and will be corrected in the thesis.

6. Next Steps

Dates	Work
November, first half	M4: per-dimension IRT router with exploration and verified escalation; stratified anchors; ablations
November, second half	E1 (onboarding curve inside the cascade) and E2 (cost against DimensionBest, LinUCB and ACRouter)
December	Live onboarding of free-tier models on the 65 web-development tasks if time allows; thesis

The prompt-dependent baselines (kNN on embeddings) will be rerun once prompt recovery covers all sources. Code clean-up and the remaining unit tests continue alongside M4.

7. Limitations

- Every task has one recorded outcome per model, so the oracle includes run-to-run noise.
- Prices for five of the eight models were converted from CNY by the dataset authors.
- The cascade costs exclude the cost of verification.
- Results cover CodeRouterBench only. The LLMRouterBench pool and our own 65 tasks are not yet in.

References

As in Report 7, plus: ICL-Router, arXiv:2510.09719; FlyRoute, arXiv:2605.22057; Routing with Generated Data (CASCAL), arXiv:2601.09692; ORACLE, arXiv:2607.22465. Data: CodeRouterBench, Hugging Face `Lance1573/CodeRouterBench`, revision 2026-06-23 (MIT). Prompt sources: DS-1000, CRUXEval, MBPP, HumanEval and BigCodeBench v0.1.4 (Hugging Face).